

BUILD GUIDE · ZERO TO MASTERY

Real-Time Weather Data Platform

Streaming ingestion, warehouse modelling, orchestration, dashboards, and a temperature forecasting model — built from scratch.

Transport	Apache Kafka
Storage	PostgreSQL
Modelling	dbt
Orchestration	Apache Airflow
Visualization	Apache Superset
Forecasting	Time-series ML
Runtime	Docker Compose

11 phases · ~60-90 hours of focused work · every line of code written by you

Contents

Start	How to use this guide	3
Design	The architecture you are building	5
Phase 0	Foundations — repo, tooling, environment	7
Phase 1	The warehouse — PostgreSQL in Docker	9
Phase 2	The source — understanding Open-Meteo	11
Phase 3	The backbone — Kafka and the producer	13
Phase 4	The sink — consumer into PostgreSQL	16
Phase 5	Transformation — dbt staging and marts	18
Phase 6	Orchestration — Apache Airflow	20
Phase 7	Visualization — Apache Superset	22
Phase 8	Forecasting — the time-series model	23
Phase 9	Hardening — tests, CI, observability	26
Phase 10	Shipping — documentation and portfolio	27
App. A	Concept glossary	28
App. B	Traps that break this exact project	29
App. C	Interview questions to test yourself	32
App. D	Stretch goals once it works	33

START HERE

How to use this guide

This guide never gives you the finished project. It gives you the goal, the concepts, the tasks, a way to prove each piece works, and the traps that will otherwise cost you an evening. The code is yours to write — that is the entire point.

The rules

1. **One phase, one branch, one merge.** `git checkout -b phase-3-kafka`, build, verify, merge to `main`. Your commit history becomes the proof that you built this.
2. **Do not skip ahead.** Each phase depends on the one before it being genuinely working, not "mostly working".
3. **Do not move on until the Definition of Done box is fully ticked.** Half-finished layers compound into a project you cannot debug.
4. **Type the code, do not paste it.** Where this guide shows a snippet, it is an anchor for something unguessable (a Docker listener config, a Kafka setting). Application logic is described, not written — you write it.
5. **Answer the self-test questions out loud.** If you cannot explain it, you have copied it, and an interviewer will find that out in ninety seconds.
6. **Write the README as you go,** at the end of every phase. A README written at the end is always a lie.

What "mastered" means here

At the end you should be able to sit in front of a stranger, open a terminal, run one command, show a live dashboard with a forecast on it, and then answer *any* question about why each component exists, what happens when it fails, and what you would change to handle a thousand cities instead of five. Appendix C is the exam.

Time budget

PHASE	FOCUS	REALISTIC TIME
0-2	Repo, Postgres, API	6-9 hours
3-4	Kafka producer and consumer	12-18 hours
5	dbt	8-12 hours
6	Airflow	8-12 hours
7	Superset	5-8 hours
8	Forecasting model	12-20 hours
9-10	Tests, CI, documentation	8-12 hours

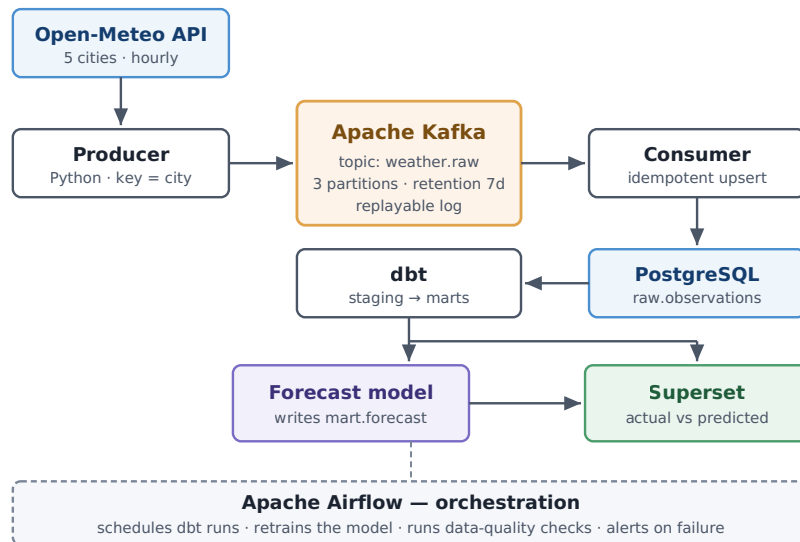
PACE

Two focused hours a day finishes this in about six weeks. Nine hours on a Saturday finishes Phase 3 and teaches you nothing that survives to Monday. Consistency beats sprints for anything with this much conceptual surface area.

DESIGN

The architecture you are building

Before writing a line of code, understand the shape of the thing. Data moves left to right through five responsibilities, and each one exists because the one before it should not be doing its job.



The target architecture. Kafka decouples ingestion from storage; Airflow drives everything that runs on a schedule rather than continuously.

Why each piece is there

COMPONENT EXISTS BECAUSE

Kafka	Ingestion and storage should fail independently. If Postgres is down for ten minutes, the producer keeps writing and the consumer catches up. It also lets you add a second consumer later without touching the first.
Postgres	Kafka is a log, not a database. You cannot join, aggregate or serve a dashboard from a topic.
dbt	Business logic belongs in version-controlled, tested SQL — not buried in a Python script or a Superset chart.
Airflow	Scheduled, dependent, retryable work with visible history. A cron job gives you none of that.
Superset	The consumer of everything above. If a stakeholder cannot see it, it did not happen.
The model	It is what turns a pipeline into a product: the platform stops describing the past and starts making a claim about the future — one you can then measure against reality.

A NOTE ON SCALE

Five cities of hourly weather is a tiny data volume, and Kafka is overkill for it in a strict engineering sense. That is fine and you should say so out loud in an interview — then explain that the architecture is chosen to demonstrate the pattern that *does* apply at ten thousand sensors, and that you know exactly which parts would change. Pretending five cities need Kafka is what makes a portfolio project look naive; knowing why it does not is what makes it look senior.

PHASE 0

Foundations

Goal: a repository a stranger could clone and contribute to Time: 2-3 h Branch: `phase-0-foundations`

You will learn

Git hygiene for data projects, why secrets must never enter history, dependency pinning, and the discipline of documenting before building.

Tasks

1. Confirm your toolchain: Docker Engine with Compose v2 (`docker compose version`), Python 3.11+, git, and at least **8 GB of free RAM** — Kafka, Airflow, Superset and Postgres together will use 4-6 GB.
2. Create the GitHub repository and set it as the remote. Make `main` your default branch and protect it from direct pushes if you want to force yourself into the branch habit.
3. Write the first real commit: the `.gitignore`, the `README.md` skeleton, and this guide in `docs/`.
4. Create a Python virtual environment (`python -m venv .venv`). This is for your local tooling and tests; the services themselves run in containers.
5. Design your configuration strategy **now**: create `.env.example` listing every variable the project will need with placeholder values, and confirm `.env` is gitignored. Every credential in this project comes from an environment variable — no exceptions, from the first line onwards.
6. Adopt a commit convention and stick to it. `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`. It costs nothing and reads as professional.
7. Write the "Build log" checklist into your README so progress is visible from the repo's front page.

TRAP

If you ever commit a real password and then delete it in a later commit, **it is still in your history and still public**. Removing it requires rewriting history (`git filter-repo`) and rotating the credential. Get `.env` into `.gitignore` before the file exists, not after.

Verify

```
git status --short      # clean, nothing untracked that should be ignored
git log --oneline       # your commits, authored by you
cat .env.example        # exists, no real values
docker compose version  # v2.x
free -g                # confirm available memory
```

DEFINITION OF DONE

- ☐ Repository exists on GitHub with at least three meaningful commits
- ☐ `.env` is ignored, `.env.example` is committed
- ☐ README shows the target architecture and the phase checklist
- ☐ You can state, in one sentence each, what Kafka, dbt, Airflow and Superset do

PHASE 1

The warehouse

Goal: a persistent, healthy PostgreSQL you can connect to Time: 3–4 h Branch: `phase-1-postgres`

You will learn

Docker Compose service definitions, named volumes versus bind mounts, healthchecks and dependency conditions, container networking and DNS, and warehouse schema layering.

Tasks

1. Write `docker-compose.yml` with a single `postgres` service. Pin the image to a specific minor version — never `latest`.
2. Pass credentials via `environment` read from your `.env`. Compose picks up a root `.env` automatically; `${POSTGRES_PASSWORD}` in the compose file is then substituted at parse time.
3. Persist data with a **named volume**, not a bind mount into the repo folder. A bind-mounted `postgres/data/` creates root-owned files inside your project that are painful to delete and easy to accidentally commit.
4. Add a healthcheck using `pg_isready`. Every later service will depend on this being healthy, and getting it right now saves you an entire class of race-condition bugs.
5. Write `postgres/init/01_schemas.sql` to create your schema layers. Files placed in `/docker-entrypoint-initdb.d/` run once, on first initialisation of an empty data directory.
6. Decide your layering and write it down in the README. A conventional choice: `raw` (exactly what arrived, never modified), `staging` (cleaned and typed), `marts` (business-ready). dbt will own staging and marts; the consumer owns raw.
7. Design the `raw.weather_observations` table. Think hard about these columns and be able to justify each: `city`, `latitude`, `longitude`, `observed_at` (`TIMESTAMPTZ`), `temperature_c`, `wind_speed_kmh`, `weather_code`, `ingested_at`, and the raw JSON payload.
8. Add a **unique constraint on (city, observed_at)**. This is the foundation of exactly-once semantics in Phase 4 — without it you cannot deduplicate on write.

DESIGN DECISION WORTH UNDERSTANDING

Store `observed_at` as `TIMESTAMPTZ`, always in UTC, and convert to local time only at the presentation layer. Mixing naive local timestamps from five cities into one column is a bug that will surface in Phase 8 as a model that mysteriously cannot learn the daily temperature cycle. Keeping the original API JSON in a `JSONB` column costs almost nothing and means a schema mistake today does not lose data forever.

Anchor: the healthcheck shape

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
  interval: 5s
  timeout: 5s
  retries: 20
  start_period: 10s
```

Note the doubled `$$` — that escapes the variable so Compose passes it through to be expanded inside the container rather than substituting it from your shell.

Verify

```
docker compose up -d
docker compose ps           # postgres shows (healthy), not just running
docker compose exec postgres psql -U $POSTGRES_USER -d $POSTGRES_DB -c '\dn'
docker compose down && docker compose up -d  # data must survive this
```

DEFINITION OF DONE

- ☐ `docker compose up -d` gives a container reporting **healthy**
- ☐ Your schemas exist and the raw table has its unique constraint
- ☐ Data survives `docker compose down` and back up
- ☐ No credential appears anywhere in a tracked file
- ☐ You can explain the difference between a named volume and a bind mount

PHASE 2

The source

Goal: fetch and normalise real weather data — no Kafka yet Time: 3–4 h Branch: `phase-2-source`

You will learn

API contracts, defensive HTTP clients, retries with backoff, data normalisation, and why you always explore a source before you stream it.

Understand the source first

Open-Meteo is free, needs no key, and has three endpoints that matter to you. Spend twenty minutes in a browser or with `curl` before writing anything:

ENDPOINT	GIVES YOU	USED IN
<code>api.open-meteo.com/v1/forecast</code>	<code>current</code> conditions and an hourly forecast	Phases 3–4 (live stream)
<code>archive-api.open-meteo.com/v1/archive</code>	ERA5 reanalysis — years of real hourly history	Phase 8 (training data)
<code>geocoding-api.open-meteo.com</code>	city name → coordinates	Optional, for configurable cities

PLAN AHEAD

The archive endpoint is the single most important thing on this page. A model trained on the handful of days your pipeline has been running will be worthless. In Phase 8 you will backfill **two or more years** of hourly history from the archive API and train on that. Know now that it exists so you design your table to hold it.

Tasks

1. Fetch one city by hand with `curl` and read the JSON properly. Identify: where the timestamp lives, what timezone it is in, what `utc_offset_seconds` means, and the units of each measure.
2. Understand WMO weather codes. The API returns an integer (0, 3, 61, 95...); you need a mapping to human labels. Write it as a module-level dictionary and handle unknown codes without crashing.
3. Write `ingestion/producer/weather_api.py`: a function that takes a city definition and returns a **normalised dictionary** — your own schema, not the API's. Decoupling your internal contract from the vendor's payload is the lesson here.
4. Make the HTTP client defensive: an explicit timeout, `raise_for_status()`, and retry with exponential backoff on 5xx and connection errors. A network call without a timeout will eventually hang your producer forever.
5. Define your cities in configuration, not in code — a YAML or JSON file listing name, latitude, longitude. Adding a sixth city must not require editing Python.
6. Write a throwaway script that fetches all cities and prints the normalised records. Confirm the numbers look like actual weather.

7. Add structured logging (the `logging` module, not `print`). Every long-running process in this project needs logs you can read at 2 a.m.

BE A GOOD CITIZEN

Open-Meteo is free and rate-limited on trust. Poll no faster than every 5–10 minutes; the underlying data only updates about every 15 minutes anyway, so a 10-second poll gives you nothing but duplicate rows and a banned IP.

DEFINITION OF DONE

- ☐ One function returns a clean, normalised record for any configured city
- ☐ Cities live in a config file; adding one requires no code change
- ☐ Timeouts and retries are in place and you have tested them (try a bad URL)
- ☐ You can explain what the API returns in UTC versus local time
- ☐ Logging is structured and useful

PHASE 3

The backbone — Kafka

Goal: weather records flowing into a Kafka topic, provably Time: 6–9 h Branch: `phase-3-kafka`

This is the phase people get wrong, and the phase that makes the project worth showing. Slow down here.

Concepts you must actually understand

TERM	WHAT IT MEANS AND WHY YOU CARE
Topic	A named, append-only log. Not a queue — messages are not removed when read.
Partition	The unit of parallelism and of ordering. Order is guaranteed <i>within</i> a partition only, never across partitions.
Message key	Determines the partition by hashing. Keying by city means all of one city's readings land in one partition and stay in order — which you want.
Offset	A message's position in a partition. Consumers track offsets; that is how they resume.
Consumer group	A set of consumers sharing the work. Each partition goes to exactly one member. More consumers than partitions means idle consumers.
Retention	How long messages live regardless of whether they were read. This is what makes replay possible.
KRaft	Kafka's built-in consensus, replacing ZooKeeper. Use it — ZooKeeper is removed in Kafka 4.x and any tutorial using it is dated.

Tasks

1. Add a Kafka broker to `docker-compose.yml` in **KRaft mode** (combined broker + controller, single node). Use a current image — Confluent's `cp-kafka` 7.6+ or `apache/kafka`.
2. Get the listener configuration right (see the anchor below). This is the number one reason Kafka-in-Docker tutorials fail.
3. Add a Kafka UI container (`provectuslabs/kafka-ui` or `redpanda-data/console`). Being able to see your topics, partitions, messages and consumer lag will save you hours and makes an excellent screenshot for your README.
4. Create the topic `weather.raw` explicitly — 3 partitions, replication factor 1 (single broker), retention 7 days. Do not rely on auto-creation; auto-created topics get default settings you did not choose.
5. Write the producer: loop over configured cities, fetch via your Phase 2 module, and produce one message per city per poll. **Key the message by city name.**
6. Serialize to JSON with UTF-8 encoding. Include a schema version field in the payload — future you will change the shape and will want to know which version a message is.
7. Configure the producer for durability: `acks=all` and `enable.idempotence=true`. Understand what each protects against.
8. Handle delivery asynchronously with a delivery callback, and call `flush()` before exit. A producer that exits without flushing silently drops buffered messages.

9. Containerize the producer: a small `Dockerfile`, added to Compose, depending on Kafka being healthy, running its loop with a configurable interval.
10. Handle `SIGTERM` gracefully so `docker compose down` does not kill it mid-batch.

Anchor: the listener configuration

Kafka tells clients where to reach it via *advertised listeners*. A client inside the Docker network must be told a different address than one on your laptop. You need two:

```
KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,EXTERNAL://0.0.0.0:29092,CONTROLLER://0.0.0.0:9093
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,EXTERNAL://localhost:29092
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
```

Containers connect to `kafka:9092`. Your laptop connects to `localhost:29092` (mapped in `ports:`). On a single-node broker the replication-factor overrides are mandatory — internal topics default to 3 and the broker will refuse to start without them.

TRAP

If your producer connects but every send times out with "no broker available", it is almost always advertised listeners: the client resolved the broker, asked it for metadata, and got back an address it cannot reach. Read what the broker advertised in the Kafka UI and ask yourself whether the client can resolve that hostname.

Verify

```
# topic exists with the settings you chose
docker compose exec kafka kafka-topics.sh --bootstrap-server kafka:9092 \
  --describe --topic weather.raw

# watch messages arrive live
docker compose exec kafka kafka-console-consumer.sh --bootstrap-server kafka:9092 \
  --topic weather.raw --from-beginning --property print.key=true
```

DEFINITION OF DONE

- ☐ `kafka-console-consumer` prints real weather JSON, keyed by city
- ☐ The topic has exactly the partitions and retention you specified
- ☐ The producer runs in a container, survives a broker restart, and reconnects
- ☐ You can explain why messages are keyed by city and what breaks if they are not
- ☐ You can explain the difference between `acks=1` and `acks=all`

SELF-TEST

- 1. Your topic has 3 partitions and you run 5 consumers in one group. What are the other two doing?*
- 2. You need strict global ordering of all weather events. What must the topic look like, and what do you give up?*
- 3. A consumer was offline for a day. Does it lose the messages? What determines the answer?*

PHASE 4

The sink

Goal: every message durably in Postgres, exactly once, no duplicates Time: 6–9 h Branch: `phase-4-consumer`

You will learn

Delivery semantics, offset management, idempotent writes, batching, dead-letter queues, and consumer lag — the concepts that separate someone who has read about Kafka from someone who has run it.

The central idea

Kafka gives you **at-least-once** delivery by default: if your consumer crashes after writing to Postgres but before committing its offset, it will re-read those messages on restart. You cannot prevent this. What you do instead is make the write **idempotent**, so processing the same message twice produces the same database state as processing it once. That is what your `(city, observed_at)` unique constraint from Phase 1 is for.

Tasks

1. Write `ingestion/consumer/`: subscribe to `weather.raw` with an explicit `group.id` and `auto.offset.reset=earliest`.
2. Turn **off** auto-commit (`enable.auto.commit=false`). Commit manually, *after* the database write succeeds. This ordering is the entire correctness argument — write it as a comment in your code so you remember why.
3. Implement the idempotent write: `INSERT ... ON CONFLICT (city, observed_at) DO UPDATE` (or `DO NOTHING`). Decide which you want and be able to justify it.
4. Batch: accumulate up to N messages or T seconds, then write them in one transaction and commit offsets once. Per-message commits will work at this volume and are the wrong habit.
5. Validate every message before writing. Missing field, wrong type, temperature of 900°C — reject it. Never let a malformed message reach your warehouse.
6. Add a dead-letter topic `weather.dlq` . A message that fails validation or repeatedly fails to write goes there with an error reason attached, and the consumer moves on. A consumer that crash-loops on one bad message is a stalled pipeline.
7. Handle Postgres being unavailable: retry with backoff, and **do not commit the offset**. When the database comes back, Kafka replays exactly what you missed. Test this by stopping the Postgres container while the consumer runs.
8. Containerize it and add it to Compose, depending on both Kafka and Postgres.
9. Log throughput and consumer lag periodically so you can see health at a glance.

THE TEST THAT PROVES YOU UNDERSTAND IT

With everything running: `docker compose stop postgres`, wait five minutes, then `docker compose start postgres`. Consumer lag should climb, then drain to zero, and your row count should be exactly what it would have been with no outage — no gaps and no duplicates. Screenshot the lag graph. That single screenshot demonstrates more competence than the rest of the dashboard.

Verify

```
-- no duplicates, ever
SELECT city, observed_at, COUNT(*) FROM raw.weather_observations
GROUP BY 1,2 HAVING COUNT(*) > 1;          -- must return zero rows

-- data is arriving continuously
SELECT city, MAX(observed_at), COUNT(*) FROM raw.weather_observations GROUP BY 1;

-- consumer lag
docker compose exec kafka kafka-consumer-groups.sh --bootstrap-server kafka:9092 \
  --describe --group weather-sink
```

DEFINITION OF DONE

- ☐ The duplicate query returns zero rows after a deliberate consumer restart
- ☐ Killing Postgres for five minutes loses no data
- ☐ A deliberately malformed message lands in the DLQ and does not stop the consumer
- ☐ Offsets are committed only after a successful write, and you can explain why
- ☐ You can define at-most-once, at-least-once and exactly-once, and say which you have

PHASE 5

Transformation with dbt

Goal: tested, documented, layered models built from raw Time: 8–12 h Branch: `phase-5-dbt`

You will learn

Analytics engineering: sources, refs and the DAG, materializations, incremental models, data tests, and documentation as code.

Tasks

1. Initialise a dbt project in `dbt/` targeting Postgres. Delete every piece of starter scaffolding you are not using — leftover example models and template comments in `dbt_project.yml` are the clearest sign of a copied project.
2. Configure `profiles.yml` to read credentials from environment variables with `{{ env_var('POSTGRES_PASSWORD') }}`. Never hardcode.
3. Declare your source in `models/sources.yml` pointing at `raw.weather_observations`, and add a `freshness` block — `warn_after: 1 hour`, `error_after: 3 hours`. Then `dbt source freshness` becomes a real pipeline health check.
4. Build the staging layer: cast types, rename to your naming convention, convert units if needed, map weather codes to labels, and deduplicate defensively. One staging model per source table, materialized as a view.
5. Build the marts. At minimum: `fct_observations` (one row per city per hour, dashboard-ready), `agg_daily_weather` (daily min/max/avg per city), and `dim_cities`. Materialize marts as tables.
6. Make `fct_observations` **incremental**. This is the most valuable dbt skill in the project. Use `materialized='incremental'`, a `unique_key`, and an `{% if is_incremental() %}` filter that only scans recent rows. Understand why full refreshes every five minutes do not scale.
7. Add tests generously: `not_null` and `unique` on keys, `accepted_values` on the weather-condition label, `relationships` from fact to dimension, and at least one custom singular test in `tests/` (for example, temperature within physically plausible bounds).
8. Document every model and column in the schema YAML. Run `dbt docs generate` and browse the lineage graph — screenshot it for your README.
9. Run dbt in a container so it works identically from Airflow in Phase 6.

TRAP

Setting `unique_key` on a model materialized as `table` does nothing at all — it is only meaningful for `incremental` (and snapshots). Seeing `materialized='table'` next to `unique_key` is a classic copied-from-a-tutorial tell. Know which config belongs to which materialization.

THINK ABOUT IT

Your incremental filter needs care. Filtering on `observed_at > (SELECT MAX(observed_at) FROM {{ this }})` silently drops late-arriving data. A safer pattern is a lookback window — reprocess the last few hours every run and let `unique_key` handle the overlap. Late data is a real phenomenon in streaming, and handling it is a senior-level detail worth being able to discuss.

Verify

```
dbt debug          # connection works
dbt build          # runs models AND tests together – all green
dbt source freshness # source is fresh
dbt docs generate  # lineage graph renders
```

DEFINITION OF DONE

- ☐ `dbt build` passes with zero failures and at least ten tests
- ☐ `fct_observations` is genuinely incremental — verified by a log line showing rows scanned
- ☐ Every model and column has a description
- ☐ No credentials in any dbt file
- ☐ You can explain view vs table vs incremental and when to use each

PHASE 6

Orchestration with Airflow

Goal: scheduled, retryable, observable pipeline runs Time: 8–12 h Branch: `phase-6-airflow`

The mental model

Your producer and consumer are **continuous** processes — they are not Airflow's job and must not be put in a DAG. Airflow owns everything **periodic**: running dbt, checking data quality, re-training the model in Phase 8, and alerting when something breaks. Getting this boundary right is a design judgement interviewers probe for.

Tasks

1. Add Airflow to Compose. For a single-machine project, `LocalExecutor` with a Postgres metadata database is the right choice — `CeleryExecutor` adds a broker and workers you do not need.
2. Give Airflow its **own database** (or at least its own schema) in Postgres, separate from your warehouse data. Mixing application metadata with analytics data is a smell.
3. Get the metadata connection variable name right — see the trap below. Then run `airflow db migrate` before starting the scheduler.
4. Do not use `airflow standalone`. It is a development convenience that generates a random admin password to a file and runs everything in one process. Define the API server / webserver and scheduler as separate Compose services and create the admin user explicitly.
5. Write your first DAG: run dbt on a schedule. Set `catchup=False` as a **DAG argument** and give tasks `retries` and `retry_delay` in `default_args`.
6. Choose how to invoke dbt and justify it. Options: `BashOperator` with dbt installed in the image (simple, couples dependencies), `DockerOperator` against the dbt image (clean isolation, needs the Docker socket mounted — a real security consideration you should be able to discuss), or Cosmos (turns each dbt model into its own Airflow task; the most impressive and the most moving parts).
7. Add a data-quality DAG: check source freshness, verify no duplicates, verify every city reported in the last hour. Fail loudly when it does not.
8. Add a sensor or short-circuit so transformations do not run when there is no new data to transform.
9. Configure failure notification — at minimum an `on_failure_callback` that logs distinctly; a Slack or email hook if you want the extra credit.
10. Store Postgres credentials as an Airflow **Connection**, not as literals in DAG code.

TRAP THAT COSTS PEOPLE A FULL DAY

The metadata database variable is `AIRFLOW__DATABASE__SQL_ALCHEMY_CONN`. The old `AIRFLOW__CORE__SQL_ALCHEMY_CONN` moved to the `[database]` section in Airflow 2.3 and no longer works in Airflow 3 — it is silently ignored, Airflow falls back to SQLite inside the container, everything appears to work, and then all your DAG history vanishes when the container restarts. If `airflow db check` does not report your Postgres, you have this bug.

TRAP

`default_args` are passed to **operators**, not to the DAG. Putting `catchup`, `description` or `schedule` in there does nothing. With a `start_date` months in the past and a five-minute schedule, an unintended catchup means tens of thousands of queued runs.

Verify

```
docker compose exec airflow-scheduler airflow db check      # points at Postgres
docker compose exec airflow-scheduler airflow dags list
docker compose exec airflow-scheduler airflow dags test <dag_id> 2026-01-01
# then in the UI: trigger it, watch it succeed, break dbt on purpose, watch it retry
```

DEFINITION OF DONE

- ☐ Airflow metadata lives in Postgres and survives a full restart
- ☐ At least two DAGs: transformation and data quality
- ☐ You have deliberately broken a task and watched retries then failure notification
- ☐ No secrets in DAG files — connections are used
- ☐ You can explain why the Kafka consumer is not an Airflow task

PHASE 7

Visualization with Superset

Goal: a dashboard that reproduces itself from version control Time: 5–8 h Branch: `phase-7-superset`

Tasks

1. Add Superset to Compose with its own metadata database and a Redis cache. Set a real `SUPERSET_SECRET_KEY` from your `.env` — the default is a known public value.
2. Initialise it properly: `superset db upgrade`, create the admin user, `superset init`. Do this in a one-shot init service so a fresh clone works without manual steps.
3. Connect Superset to your warehouse and register your dbt marts as datasets. Point charts at **marts**, never at raw tables — that boundary is the reason the marts layer exists.
4. Build the dashboard. Aim for six charts that answer real questions, not six charts that fill space:
 - current temperature and wind, as big-number KPIs with a trend sparkline
 - 24-hour temperature trend, one line per city
 - temperature comparison across cities
 - distribution of weather conditions
 - latest observations table
 - (after Phase 8) **forecast versus actual** — the centrepiece
5. Enable auto-refresh so the dashboard visibly updates while someone is watching it. For a project called "real-time", this matters.
6. **Export the dashboard as YAML and commit it.** Then wire the import into your init service so a fresh `docker compose up` produces the dashboard with no clicking. This is what makes it reproducible infrastructure rather than a manual artifact.
7. Test the reproducibility claim honestly: `docker compose down -v`, then `up`, and confirm the dashboard is there.

DEVELOPMENT MODE IS NOT PRODUCTION

Most Superset compose examples run `flask run --reload --debugger` with `FLASK_DEBUG=true` and `admin/admin`. That is a development server with a remote debugger exposed. Fine for local work — but say so in your README, and know that `gunicorn`, a strong secret key and a real password are what production needs. Knowing the difference is the point.

DEFINITION OF DONE

- ☐ `docker compose down -v && docker compose up -d` recreates the dashboard automatically
- ☐ Dashboard assets are committed as YAML
- ☐ Charts read from marts only
- ☐ Auto-refresh works and the data visibly moves
- ☐ The secret key and admin password come from environment variables

PHASE 8

Forecasting — your signature addition

Goal: a model that predicts the next 24 hours and beats a real baseline Time: 12–20 h Branch: `phase-8-forecast`

This phase is what makes the project yours rather than another Airflow-plus-dbt demo. It is also where most people quietly cheat — by evaluating wrongly, or by never comparing against a baseline. Do it properly and it becomes the strongest thing on your CV.

Step 1 — Get real training data

Your pipeline has, at best, days of history. A temperature model needs to see full daily cycles and seasonal variation. Backfill from the Open-Meteo **archive** endpoint:

- Pull at least **two years** of hourly history per city — roughly 17,500 rows per city, 87,000 rows total. Trivial for Postgres and enough to learn daily and seasonal structure.
- Write it through a one-off backfill script into `raw.weather_observations`, reusing your normalisation so the schema matches exactly.
- Mark backfilled rows with a `source` column (`'archive'` vs `'stream'`). You will want to distinguish them later, and honest data lineage matters.

Step 2 — Establish baselines before any model

A forecast is only meaningful relative to something trivial. Implement two and record their error:

BASELINE	PREDICTION FOR TIME T+H	WHY IT MATTERS
Persistence	the temperature at time t	Astonishingly hard to beat at 1–3 hours ahead
Seasonal naive	the temperature at t+h minus 24 hours	Captures the daily cycle for free; the real bar to clear at 24h
Climatology	the historical mean for that city, month and hour	The long-horizon floor

THE HONESTY RULE

If your fancy model does not beat seasonal naive, **you do not have a model, you have a slow way of being wrong**. Report this if it happens — an engineer who measures and reports a negative result is far more employable than one who reports an unverifiable win.

Step 3 — Frame the problem

- **Target:** temperature in °C. **Horizon:** 24 hours ahead, hourly. **Granularity:** per city.
- Decide between **direct** multi-horizon (24 models, one per lead time) and **recursive** (one model applied iteratively, feeding its own predictions back in). Direct is more robust — recursive compounds its own errors. Know the trade-off.

- Handle gaps explicitly. A missing hour is not zero. Reindex to a complete hourly grid per city and decide, deliberately, whether to interpolate short gaps and drop long ones.

Step 4 — Features

- **Lags** of temperature: t-1, t-2, t-3, t-24, t-48, t-168 (one week).
- **Rolling statistics**: mean, min, max, standard deviation over 3, 24 and 168 hour windows.
- **Cyclical time encoding**: hour-of-day and day-of-year as `sin / cos` pairs. Encoding hour as the integer 0-23 tells the model that 23:00 and 00:00 are 23 units apart, which is false and hurts.
- **Other measures**: wind speed and weather code, lagged the same way.
- **City** as a categorical feature if you train one shared model.

LEAKAGE — READ THIS TWICE

Every feature must be computable at prediction time using **only past data**. Two ways this project invites you to cheat: (1) a rolling window that includes the current row, and (2) using Open-Meteo's own forecast fields as inputs. The second is the subtle one — the API will happily hand you the future, and a model fed the answer will look brilliant and be worthless. Use the API's forecast as a *benchmark to compare against*, never as a feature. That comparison, incidentally, is a great section for your README.

Step 5 — Evaluate correctly

THE MISTAKE THAT INVALIDATES EVERYTHING

Never use a random train/test split on time series. Randomly holding out rows lets the model train on Tuesday and Thursday to predict Wednesday — information it will never have in production. Your error will look excellent and mean nothing. Split chronologically, always.

- Use **walk-forward (expanding-window) backtesting**: train on everything up to time T, predict the next 24 hours, roll T forward, repeat across many origins. Average the errors.
- Report **MAE** and **RMSE** in °C, broken down *by lead time* — error at hour 1 and hour 24 are completely different quantities and a single number hides that. Avoid MAPE: temperatures near 0°C make it explode.
- Plot predicted versus actual for a held-out week. Your eyes will catch systematic problems no metric will.
- Put the comparison table — baselines versus your model versus Open-Meteo's own forecast — directly in your README.

Step 6 — Choose a model

APPROACH	VERDICT
SARIMA (statsmodels)	Classical, interpretable, per-city. Great for the write-up because it forces you to understand seasonality and stationarity. Slow to fit, awkward with exogenous features.
Holt-Winters	Simplest thing that models level, trend and seasonality. Worth ten minutes as a second baseline.
Gradient boosting on lag features (LightGBM / <code>HistGradientBoostingRegressor</code>)	Recommended. Handles multiple cities and exogenous features in one model, trains in seconds, and is what most companies actually ship. All the intellectual work is in your feature engineering, which is where it should be.
Prophet	Easy and readable output, decent at daily/seasonal structure. Heavy dependency and rarely the best performer.
LSTM / GRU	Only after the above work. With 87k rows it will probably lose to gradient boosting. Worth doing as a deliberate experiment you then report honestly.

Step 7 — Serve the predictions

1. Create `marts.weather_forecast`: city, forecast_made_at, target_time, horizon_hours, predicted_temp_c, model_version. Storing `forecast_made_at` is what later lets you measure how good your forecasts actually were.
2. Write an inference script that loads the trained model, builds features for the latest data, predicts the next 24 hours per city, and upserts into that table.
3. Add two Airflow DAGs: **inference** hourly, and **retraining** weekly. Version each model artifact and never overwrite the previous one.
4. Build a **forecast accuracy** model in dbt that joins past forecasts to what actually happened, and chart rolling MAE by lead time in Superset. This closes the loop and is genuinely rare in portfolio projects.
5. Add a drift check: if rolling MAE degrades beyond a threshold, fail the DAG loudly.

DEFINITION OF DONE

- ☐ Two or more years of history backfilled from the archive API
- ☐ Three baselines implemented and their errors recorded
- ☐ Walk-forward backtest, with MAE and RMSE reported per lead time
- ☐ Your model beats seasonal naive at 24 hours — or you say clearly that it does not
- ☐ Forecasts are written to a table and charted against actuals in Superset
- ☐ Airflow retrains on a schedule and monitors accuracy drift
- ☐ You can explain, without notes, why random splits are invalid here

PHASE 9

Hardening

Goal: the parts that separate a demo from an engineered system Time: 5–8 h Branch: `phase-9-harden-ing`

Tasks

1. **Unit tests** with pytest: normalisation logic, weather-code mapping, feature engineering, validation rules. Mock the HTTP calls — a test that hits the real API is not a unit test and will fail on a train.
2. **Integration test**: spin up Kafka and Postgres, produce a known message, assert it lands correctly in the database. `testcontainers` makes this straightforward.
3. **CI** in `.github/workflows/`: on every push, run linting (`ruff`), formatting checks, pytest, and `dbt parse/dbt compile`. A green badge on your README is worth real points.
4. **Pre-commit hooks**: `ruff`, trailing whitespace, and a secret scanner (`detect-secrets` or `git-leaks`).
5. **Structured JSON logging** across all services with consistent fields.
6. **Health endpoints or heartbeats** for the producer and consumer, so "is it running" is answerable without reading logs.
7. **Resource limits** in Compose (`mem_limit`) so one runaway container cannot take down your laptop.
8. **A Makefile**: `make up` , `make down` , `make test` , `make logs` , `make reset` . Small, and it signals care.
9. **Document failure modes** in the README: what happens when the API is down, Kafka is down, Postgres is down, the model is stale. Being able to answer this is a senior trait.

DEFINITION OF DONE

- ☐ CI runs on every push and is green
- ☐ Test coverage on the logic that matters (not a coverage percentage — the right tests)
- ☐ Pre-commit blocks a deliberately committed fake secret
- ☐ A failure-modes section exists in the README

PHASE 10

Shipping it

Goal: someone can understand and run this without you in the room Time: 3-4 h Branch: `phase-10-docs`

Tasks

1. **Rewrite the README from scratch**, now that you know what you built. Structure: one-line description, architecture diagram, screenshots, quick start, what each component does, the forecast accuracy table, failure modes, what you would do differently, and what you would change to scale it.
2. **Draw a real architecture diagram** (Excalidraw, draw.io, Mermaid). Export a PNG and commit it.
3. **Screenshots**: the Superset dashboard, the Airflow DAG graph, the Kafka UI showing lag draining, the dbt lineage graph. Four images that prove every layer works.
4. **A short demo GIF or video** — `docker compose up` to live dashboard. This gets watched when a README does not get read.
5. **Test the cold start yourself**: clone into a fresh directory, copy `.env.example` to `.env`, run one command, and follow your own instructions literally. Every friction point you hit, a recruiter hits too.
6. **Add a LICENSE** (MIT is fine).
7. **Write the "what I would do differently" section**. Honest limitations — single broker, no schema registry, dev-mode Superset, small training set — read as maturity, not weakness.
8. **Write it up** as a blog post or a long LinkedIn post. Explaining it publicly is the final compression step that turns "I built this" into "I understand this".

HOW TO CLAIM IT HONESTLY

You built it: you wrote the code, made the design decisions, and can defend them. Guidance is not authorship — every engineer works from documentation, blog posts and colleagues. What makes it yours is that the commit history is real, you hit the bugs, you fixed them, and you can be dropped into any file and explain what it does and why. That is the standard. Meet it and say so without hedging.

DEFINITION OF DONE

- ☐ A stranger goes from clone to live dashboard using only your README
- ☐ Architecture diagram and four screenshots committed
- ☐ Forecast accuracy table published with real numbers
- ☐ Limitations documented honestly
- ☐ Every box in the build log is ticked

APPENDIX A

Concept glossary

TERM	MEANING
Idempotent	Doing it twice has the same effect as doing it once. The core defence against at-least-once delivery.
At-least-once	Every message is delivered, possibly more than once. Kafka's practical default.
Exactly-once	Every message has effect precisely once. Achieved end-to-end in this project by combining at-least-once delivery with idempotent writes.
Consumer lag	Messages produced minus messages consumed. The single most important streaming health metric.
Backpressure	What happens when a downstream stage cannot keep up. Kafka absorbs it in the log rather than dropping data.
Dead-letter queue	Where unprocessable messages go so one bad record cannot stall the pipeline.
Medallion / layering	raw → staging → marts. Each layer has one job and one owner.
Slowly changing dimension	A dimension whose attributes change over time; dbt snapshots capture the history.
Idempotent DAG	An Airflow run that produces the same result if re-run for the same interval. Required for retries to be safe.
Backfill	Running a pipeline over historical periods it never ran for.
Data leakage	Training on information unavailable at prediction time. Silently inflates every metric.
Walk-forward validation	Chronological backtesting with an expanding training window. The only honest evaluation for forecasting.
Horizon / lead time	How far ahead a prediction reaches. Error grows with it, so always report per horizon.
Model drift	Accuracy decaying as the world changes. Detected by tracking error over time, which is why you store forecasts.

APPENDIX B

Traps that break this exact project

Every one of these has been observed in real implementations of this pipeline. Read this page before starting the phase it belongs to, and again when something inexplicable happens.

PHASE TRAP

0	Committing a real <code>.env</code> . It stays in history forever; the credential must be rotated.
1	Bind-mounting Postgres data into the repo. Root-owned files you cannot delete and might commit. Use a named volume.
1	No healthcheck, so dependent services start before Postgres accepts connections and fail with a confusing error.
2	HTTP calls with no timeout. The producer hangs forever with no error and no data.
2	Storing naive local timestamps. Breaks cross-city comparison and quietly ruins Phase 8.
3	Advertised listeners. Wrong values mean clients connect and then time out. Cause of most Kafka-in-Docker failures.
3	Single-node broker without <code>OFFSETS_TOPIC_REPLICATION_FACTOR=1</code> — the broker refuses to start.
3	Producer exits without <code>flush()</code> . Buffered messages vanish silently.
3	Relying on topic auto-creation, then wondering why you have one partition and infinite retention.
4	Auto-commit left on. Offsets commit before the write succeeds; a crash means permanent data loss.
4	Committing offsets when the database write failed. Same outcome, but you did it deliberately.
4	No unique constraint, so a replay duplicates rows and every aggregate is silently wrong.
4	No DLQ — one malformed message crash-loops the consumer and the pipeline stops.
5	<code>unique_key</code> on a <code>table</code> materialization. Does nothing; a copied-tutorial tell.
5	Full table rebuilds every five minutes. Works at five cities, absurd at scale, and interviewers ask.
5	Column names in <code>sources.yml</code> that do not match reality (<code>weather_descriptions</code> vs <code>weather_description</code>). Silent until a test needs it.
5	Leaving dbt starter scaffolding in <code>dbt_project.yml</code> and the generated README.
6	<code>AIRFLOW_CORE_SQL_ALCHEMY_CONN</code> instead of <code>AIRFLOW_DATABASE_SQL_ALCHEMY_CONN</code> . Silently falls back to SQLite.
6	<code>catchup</code> inside <code>default_args</code> , where it has no effect. Risk of tens of thousands of backfill runs.
6	<code>airflow standalone</code> in a compose file presented as a deployment. It is a dev tool.
6	Mounting <code>/var/run/docker.sock</code> without acknowledging it grants effective host root.
6	No <code>retries</code> in <code>default_args</code> , so one transient network blip is a hard failure.
7	Default <code>SUPERSET_SECRET_KEY</code> . It is public and it signs session cookies.
7	Charts built by clicking and never exported. The dashboard is not reproducible and dies with the volume.
7	Charts pointed at raw tables, bypassing the modelling layer entirely.
8	Random train/test split. Invalidates every number you report.
8	Training on a few days of streamed data instead of backfilled history.

8	Feeding the API's own forecast into the model. Looks brilliant, predicts nothing.
8	Rolling windows that include the current row. Subtle, common, fatal.
8	Reporting one error number across all horizons, hiding that hour 24 is far worse than hour 1.
8	No baseline comparison, so "MAE 1.8°C" means nothing to anyone.
9	Tests that call the live API. Slow, flaky, and useless offline.
10	A README whose quick start you never actually followed on a clean clone.

APPENDIX C

Interview questions to test yourself

Answer these out loud, from memory, without opening the code. If you stumble, that part is not yet yours.

Architecture

1. Walk me through your pipeline end to end in two minutes.
2. Why Kafka? What would you use instead for five cities, and why did you still choose Kafka?
3. What happens if Postgres is down for an hour? For a week?
4. What changes if you go from 5 cities to 50,000 sensors reporting every second?
5. Which component is your single point of failure and how would you remove it?

Streaming

1. Explain partitions, keys and ordering guarantees.
2. How do you achieve effectively-exactly-once here? Where precisely does that guarantee live?
3. Your consumer lag is growing steadily. Walk me through your diagnosis.
4. Why do you commit offsets after the write rather than before?
5. What is in your dead-letter queue, and who looks at it?

Modelling and orchestration

1. Why layer raw, staging and marts? What breaks if you skip staging?
2. How does your incremental model handle late-arriving data?
3. Which dbt tests do you have and which real bug would each have caught?
4. Why is the Kafka consumer not an Airflow task?
5. What makes an Airflow task safe to retry, and is yours?

Machine learning

1. Why can you not use a random train/test split?
2. What is your baseline and by how much do you beat it at 1 hour and at 24 hours?
3. Name three ways this project could leak future information into training.
4. Why sin/cos encoding for hour of day?
5. Direct versus recursive multi-step forecasting — which did you choose and why?
6. How would you know your model has degraded in production, and what happens then?

APPENDIX D

Stretch goals once it works

Only after every phase is done. Each of these is a weekend and each meaningfully raises the ceiling of the project.

ADDITION	WHAT IT DEMONSTRATES
Schema Registry + Avro	Schema evolution and contract enforcement between producer and consumer — the thing every real Kafka deployment has and every tutorial omits.
Kafka Streams or Flink	Real stream processing: windowed aggregations and alerting computed in flight rather than in the warehouse.
Great Expectations or Soda	Data quality as a first-class, reported, versioned concern beyond dbt tests.
MLflow	Experiment tracking and a model registry — proper ML lifecycle management rather than pickles in a folder.
FastAPI serving layer	Exposing forecasts over HTTP; turns the platform into a product with an API.
Prometheus + Grafana	Real observability: consumer lag, throughput and freshness as time series with alerts.
Terraform + a cloud deployment	Infrastructure as code and the ability to hand someone a live URL.
Anomaly detection	Flagging observations that deviate sharply from forecast — a genuinely useful feature built on what you already have.
Alerting DAG	Notify when a city stops reporting, freshness breaches, or forecast error spikes.

LAST WORD

The single most valuable thing in this entire guide is the habit of not moving on until the current layer genuinely works and you can explain it. Everything else — Kafka, dbt, Airflow, the model — is a specific technology that will be replaced within a decade. That habit will not be.